"The Future of Filmmaking"

PDF MAESTRO PARA CODEX / OPENAI

Sistema completo de producción IA powered by Kling AI API

PROYECTO	NEXFRAME FILMS
VERSIÓN	v1.0.0
BASE	Open Generative AI (fork + rediseño)
API ENGINE	Kling AI API — kling.ai
UI REFERENCE	Higgsfield.ai (reimaginado)
STACK	Next.js 14 + React 18 + Tailwind CSS
AUTOR	YANKYFILMS / Jean Carlos
CODEX TARGET	OpenAI Codex — instrucciones autónomas

INSTRUCCIONES: Lee este PDF completo antes de escribir UNA sola línea de código. Sigue las secciones en orden. Cada sección es autónoma y ejecutable. No uses placeholders. No uses botones de ejemplo. TODO debe ser 100% funcional.

SECCIÓN 01

¿QUÉ ES NEXFRAME FILMS?

NEXFRAME FILMS es un estudio de producción cinematográfica con IA, construido sobre el repositorio Open Generative AI (github.com/anil-matcha/open-generative-ai) como base técnica, con el diseño completamente reimaginado y el motor de IA reemplazado por la API oficial de Kling AI. El resultado es una plataforma que compite directamente con Higgsfield.ai pero con identidad propia, control total del dueño (YANKYFILMS) y sin ningún coste de plataforma intermediaria.

PRINCIPIOS DEL SISTEMA

- FORK TOTAL: El repositorio Open Generative AI se clona íntegro. Conserva toda su lógica interna.
- API SWAP: Muapi.ai (original) es reemplazado 100% por Kling AI API. Mismo patrón submit → poll.
- UI COMPLETO: La interfaz se rehace desde cero con el sistema de diseño NEXFRAME FILMS.
- DASHBOARD NUEVO: Panel de control propio, diferente al original, con control total del usuario.
- CERO PLACEHOLDERS: Todo botón, todo campo, toda función debe estar operativa al 100%.
- STUDIOS EXPANDIDOS: Se añaden Documentary Studio y Music Video Studio sobre la base existente.
- SCRIPT ENGINE: Módulo de generación de guiones y narrativas integrado en el dashboard.
- SELF-HOSTED READY: Preparado para correr en GRANOSCAR (i7-13700K, 96GB, RTX 4070 Ti).

CAPACIDADES DE KLING AI API

Kling AI (kling.ai) provee acceso completo a sus modelos de última generación mediante API REST. Todos los endpoints usan el patrón asíncrono: se submite una tarea y se hace polling hasta obtener el resultado.

CAPACIDAD	MODELO KLING	ENDPOINT BASE
Video Text-to-Video	Kling 1.0 / 1.5 / 2.0	/v1/videos/text2video
Video Image-to-Video	Kling 1.0 / 1.5 / 2.0	/v1/videos/image2video
Video Image-to-Video Pro	Kling 2.1 Master	/v1/videos/image2video
Imagen Text-to-Image	Kling Image	/v1/images/generations
Sonido / Audio	Kling Sound	/v1/sounds/generations
Efectos Especiales	Kling Effects	/v1/effects/

Lip Sync / Avatars	Kling Avatars	/v1/videos/lip-sync
Video Omni (multimodal)	Kling Omni 3.0	/v1/videos/text2video
Extend Video	Kling Extend	/v1/videos/video-extend
Virtual Try-On	Kling Virtual	/v1/images/kolors-virtual-try-on

SECCIÓN 02

COMANDOS DE INICIALIZACIÓN

Ejecuta estos comandos exactamente en este orden desde tu terminal:

```
# 1. Clonar el repositorio base con submódulos
git clone --recurse-submodules https://github.com/anil-matcha/open-generative-ai.git
nexframe-films

# 2. Entrar al directorio
cd nexframe-films

# 3. Instalar dependencias y compilar paquetes workspace
npm run setup

# 4. Crear archivo de variables de entorno
cp .env.example .env.local

# 5. Arrancar en modo desarrollo
npm run dev

# 6. Para app Electron (desktop)
npm run electron:dev
```

VARIABLES DE ENTORNO (.env.local)

```
# ■■■ KLING AI API ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
NEXT_PUBLIC_KLING_API_KEY=tu_api_key_de_kling_ai
NEXT_PUBLIC_KLING_API_BASE=https://api.klingai.com

# ■■■ NEXFRAME CONFIG ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
NEXT_PUBLIC_APP_NAME=NEXFRAME FILMS
NEXT_PUBLIC_APP_TAGLINE=The Future of Filmmaking
NEXT_PUBLIC_APP_VERSION=1.0.0

# ■■■ OPCIONAL: Analytics ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
NEXT_PUBLIC_GA_ID=

# NOTA: Nunca commitear .env.local al repositorio
```

DEPENDENCIAS ADICIONALES A INSTALAR

```
# Instalar dependencias adicionales para NEXFRAME FILMS
```

```
npm install framer-motion
npm install @radix-ui/react-dialog @radix-ui/react-tabs @radix-ui/react-select
npm install @radix-ui/react-slider @radix-ui/react-switch @radix-ui/react-progress
npm install lucide-react
npm install clsx tailwind-merge
npm install zustand
npm install react-hot-toast
npm install react-dropzone
npm install axios
npm install date-fns
```

SECCIÓN 03

Toda la comunicación con Kling AI sigue el patrón asíncrono: SUBMIT (POST) → POLL (GET) hasta status=succeed. La autenticación usa Bearer Token en el header Authorization.

CLIENTE API BASE

Crea el archivo src/lib/kling-client.js con el siguiente código exacto:

```
// src/lib/kling-client.js
// Cliente oficial Kling AI para NEXFRAME FILMS

const KLING_BASE = process.env.NEXT_PUBLIC_KLING_API_BASE || 'https://api.klingai.com';
const KLING_KEY = process.env.NEXT_PUBLIC_KLING_API_KEY;

function getHeaders() {
return {
'Content-Type': 'application/json',
'Authorization': `Bearer ${KLING_KEY}`,
};
}

// SUBMIT: envía una tarea y retorna task_id
async function klingSubmit(endpoint, payload) {
const res = await fetch(`${KLING_BASE}${endpoint}`, {
method: 'POST',
headers: getHeaders(),
body: JSON.stringify(payload),
});
if (!res.ok) {
const err = await res.json();
throw new Error(err.message || 'Kling API error');
}
const data = await res.json();
return data.data?.task_id;
}

// POLL: espera hasta que status === succeed o failed
async function klingPoll(pollEndpoint, taskId, maxWait = 300000) {
const start = Date.now();
while (Date.now() - start < maxWait) {
const res = await fetch(`${KLING_BASE}${pollEndpoint}/${taskId}`, {
headers: getHeaders(),
```

```
});
const data = await res.json();
const status = data.data?.task_status;
if (status === 'succeed') return data.data;
if (status === 'failed') throw new Error(data.data?.task_status_msg || 'Task failed');
await new Promise(r => setTimeout(r, 3000));
}
throw new Error('Timeout: la tarea no completó en el tiempo límite');
}

// ■■■ TEXT TO VIDEO ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export async function generateTextToVideo(params) {
const payload = {
model_name: params.model || "kling-v1",
prompt: params.prompt,
negative_prompt: params.negative_prompt || "",
cfg_scale: params.cfg_scale || 0.5,
mode: params.mode || "std",
duration: params.duration || "5",
aspect_ratio: params.aspect_ratio || "16:9",
};
const taskId = await klingSubmit('/v1/videos/text2video', payload);
return await klingPoll('/v1/videos/text2video', taskId);
}

// ■■■ IMAGE TO VIDEO ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export async function generateImageToVideo(params) {
const payload = {
model_name: params.model || "kling-v1",
image: params.image_url,
image_tail: params.image_tail || null,
prompt: params.prompt || "",
negative_prompt: params.negative_prompt || "",
cfg_scale: params.cfg_scale || 0.5,
mode: params.mode || "std",
duration: params.duration || "5",
aspect_ratio: params.aspect_ratio || "16:9",
};
const taskId = await klingSubmit('/v1/videos/image2video', payload);
return await klingPoll('/v1/videos/image2video', taskId);
}

// ■■■ TEXT TO IMAGE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export async function generateImage(params) {
const payload = {
model_name: params.model || "kling-v1",
```

```
prompt: params.prompt,
negative_prompt: params.negative_prompt || "",
image_reference: params.image_reference || null,
image_fidelity: params.image_fidelity || 0.5,
n: params.n || 1,
aspect_ratio: params.aspect_ratio || "1:1",
};
const taskId = await klingSubmit('/v1/images/generations', payload);
return await klingPoll('/v1/images/generations', taskId);
}
```

```
// ■■■ TEXT TO AUDIO ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export async function generateSound(params) {
const payload = {
prompt: params.prompt,
negative_prompt: params.negative_prompt || "",
duration: params.duration || 5,
};
const taskId = await klingSubmit('/v1/sounds/generations', payload);
return await klingPoll('/v1/sounds/generations', taskId);
}
```

```
// ■■■ EFFECTS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export async function generateEffect(params) {
const payload = {
input: {
image_url: params.image_url,
duration: params.duration || 5,
model_name: params.model || "kling-v1",
},
effect_scene: params.effect_scene,
};
const taskId = await klingSubmit('/v1/effects/human', payload);
return await klingPoll('/v1/effects/human', taskId);
}
```

```
// ■■■ LIP SYNC ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export async function generateLipSync(params) {
const payload = {
input: {
video_url: params.video_url || null,
image_url: params.image_url || null,
audio_url: params.audio_url,
mode: params.mode || "standard",
},
};
```

```
const taskId = await klingSubmit('/v1/videos/lip-sync', payload);
return await klingPoll('/v1/videos/lip-sync', taskId);
}

// ■■■ VIDEO EXTEND ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export async function extendVideo(params) {
const payload = {
video_id: params.video_id,
prompt: params.prompt || "",
duration: params.duration || "5",
};
const taskId = await klingSubmit('/v1/videos/video-extend', payload);
return await klingPoll('/v1/videos/video-extend', taskId);
}

// ■■■ VIRTUAL TRY-ON ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export async function virtualTryOn(params) {
const payload = {
human_image: params.human_image,
cloth_image: params.cloth_image,
};
const taskId = await klingSubmit('/v1/images/kolors-virtual-try-on', payload);
return await klingPoll('/v1/images/kolors-virtual-try-on', taskId);
}

// ■■■ UPLOAD FILE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export async function uploadFile(file) {
const formData = new FormData();
formData.append('file', file);
const res = await fetch(`${KLING_BASE}/v1/files/upload`, {
method: 'POST',
headers: {
'Authorization': `Bearer ${KLING_KEY}`,
},
body: formData,
});
const data = await res.json();
return data.data?.url;
}
```

MODELOS DISPONIBLES EN KLING AI

MODELO	TIPO	RESOLUCIÓN	DURACIÓN
kling-v1	T2V / I2V	720p	5s / 10s
kling-v1-5	T2V / I2V	1080p	5s / 10s

kling-v1-6	T2V / I2V	1080p	5s / 10s
kling-v2-master	T2V / I2V	1080p	5s / 10s
kling-v2-1-master	T2V / I2V	1080p	5s / 10s
kling-v3	T2V / I2V Omni	1080p	5s / 10s
kolors-v1	T2I	1024px	N/A

SECCIÓN 04

PALETA DE COLORES

Estos son los tokens de color exactos. Configúralos en tailwind.config.js:

```
// tailwind.config.js — Reemplaza el extend.colors existente
/** @type {import('tailwindcss').Config} */
module.exports = {
content: ["./src/**/*.{js,jsx,ts,tsx}", "./components/**/*.{js,jsx}",
"./app/**/*.{js,jsx}"],
theme: {
extend: {
colors: {
nf: {
black: '#0A0A0A', // Fondo principal
deep: '#0D0D0D', // Fondo profundo
dark: '#111111', // Sidebar, headers
card: '#161616', // Cards, panels
border: '#222222', // Bordes
surface: '#1A1A1A', // Inputs, areas
red: '#E02020', // Acento primario
'red-dim':'#8B0000', // Acento secundario
amber: '#F59E0B', // Warning, labels
gold: '#D4AF37', // Premium, highlights
white: '#FFFFFF', // Texto principal
grey: '#9CA3AF', // Texto secundario
'grey-dim':'#6B7280', // Texto terciario
light: '#F3F4F6', // Texto claro
cyan: '#06B6D4', // Info, endpoints
green: '#10B981', // Éxito, activo
purple: '#8B5CF6', // Especial
}
},
fontFamily: {
heading: ['Bebas Neue', 'sans-serif'],
body: ['Space Grotesk', 'sans-serif'],
mono: ['JetBrains Mono', 'monospace'],
```

```
},
animation: {
'pulse-red': 'pulse 2s cubic-bezier(0.4,0,0.6,1) infinite',
'glow': 'glow 3s ease-in-out infinite alternate',
'slide-up': 'slideUp 0.3s ease-out',
'fade-in': 'fadeIn 0.4s ease-in',
},
keyframes: {
glow: { '0%': {boxShadow:'0 0 5px #E02020'}, '100%': {boxShadow:'0 0 20px #E02020, 0 0 40px #8B000060'}},
slideUp: { from: {opacity:'0',transform:'translateY(20px)'}, to: {opacity:'1',transform:'translateY(0)'}},
fadeIn: { from: {opacity:'0'}, to: {opacity:'1'}},
},
backdropBlur: {
xs: '2px',
},
},
},
plugins: [require('@tailwindcss/forms')],
};
```

TIPOGRAFÍA

En app/layout.js añade las Google Fonts. Bebas Neue para títulos/headings y Space Grotesk para el cuerpo.

```
// app/layout.js — Google Fonts
import { Bebas_Neue, Space_Grotesk, JetBrains_Mono } from 'next/font/google';

const bebas = Bebas_Neue({
weight: '400',
subsets: ['latin'],
variable: '--font-heading',
});

const spaceGrotesk = Space_Grotesk({
subsets: ['latin'],
variable: '--font-body',
});

const jetbrains = JetBrains_Mono({
subsets: ['latin'],
variable: '--font-mono',
});
```

```
export default function RootLayout({ children }) {
return (
<html lang="es" className="dark">
<body className={`${bebas.variable} ${spaceGrotesk.variable} ${jetbrains.variable}
bg-nf-black text-nf-white font-body antialiased`}>
{children}
</body>
</html>
);
}
```

COMPONENTES BASE GLOBALES

Crea src/components/ui/ con estos componentes base:

```
// src/components/ui/Button.jsx
import { cva } from 'class-variance-authority';
import { cn } from '@/lib/utils';

const buttonVariants = cva(
'inline-flex items-center justify-center gap-2 rounded font-body font-semibold
transition-all duration-200 disabled:opacity-50 disabled:cursor-not-allowed',
{
variants: {
variant: {
primary: 'bg-nf-red text-white hover:bg-red-600 active:scale-95 shadow-lg
shadow-nf-red/20',
secondary: 'bg-nf-card border border-nf-border text-nf-white hover:border-nf-red
hover:text-nf-red',
ghost: 'text-nf-grey hover:text-nf-white hover:bg-nf-surface',
danger: 'bg-red-900 text-white hover:bg-red-700 border border-red-700',
glow: 'bg-nf-red text-white shadow-[0_0_20px_#E02020] hover:shadow-[0_0_35px_#E02020]',
},
size: {
sm: 'h-8 px-3 text-xs',
md: 'h-10 px-4 text-sm',
lg: 'h-12 px-6 text-base',
xl: 'h-14 px-8 text-lg',
icon:'h-10 w-10',
},
},
defaultVariants: { variant: 'primary', size: 'md' },
}
);
```

```
export function Button({ className, variant, size, children, ...props }) {
return (
<button className={cn(buttonVariants({ variant, size }), className)} {...props}>
{children}
</button>
);
}
```

SECCIÓN 05

El repositorio Open Generative AI tiene esta estructura. Las columnas indican qué hacer con cada parte.

ARCHIVO / DIRECTORIO	ACCIÓN	DESCRIPCIÓN
packages/studio/src/muapi.js	REEMPLAZAR	Reemplazar con src/lib/kling-client.js
packages/studio/src/models.js	MODIFICAR	Actualizar modelos con los de Kling AI
packages/studio/src/index.js	CONSERVAR	Exporta los studios — mantener estructura
packages/studio/src/components/ImageStudio.jsx	MODIFICAR	Conectar a klingClient.generateImage()
packages/studio/src/components/VideoStudio.jsx	MODIFICAR	Conectar a klingClient T2V / I2V
packages/studio/src/components/LipSyncStudio.jsx	MODIFICAR	Conectar a klingClient.generateLipSync()
packages/studio/src/components/CinemaStudio.jsx	MODIFICAR	Conservar lógica, nuevo diseño NF
packages/studio/src/components/WorkflowStudio.jsx	CONSERVAR	Mantener como está
components/StandaloneShell.js	REEMPLAZAR	Nuevo layout NEXFRAME FILMS completo
components/ApiKeyModal.js	REEMPLAZAR	Nuevo modal de API Key con diseño NF
app/layout.js	REEMPLAZAR	Nuevo layout con fuentes NF + metadata
app/page.js	MODIFICAR	Redirect a /studio o /dashboard
app/studio/page.js	REEMPLAZAR	Renderiza NexFrameShell en lugar de StandaloneShell
tailwind.config.js	REEMPLAZAR	Sistema de diseño NEXFRAME (ver Sección 04)
next.config.mjs	CONSERVAR	Mantener transpilePackages
src/lib/kling-client.js	CREAR	Cliente Kling AI (ver Sección 03)
src/lib/store.js	CREAR	Zustand store global de estado
src/lib/utils.js	CREAR	Helpers: cn(), formatDate(), etc.
src/components/ui/	CREAR	Componentes UI base (Button, Input, Card...)
src/components/layout/	CREAR	Sidebar, Header, Dashboard layout
src/components/studios/	CREAR	Wrappers NF para cada studio
src/components/dashboard/	CREAR	Widgets del panel de control
app/dashboard/page.js	CREAR	Página principal del dashboard NF
app/api/generate/route.js	CREAR	Proxy API route para Kling (ocultar key)

STORE GLOBAL (ZUSTAND)

```
// src/lib/store.js
import { create } from 'zustand';
```

```
import { persist } from 'zustand/middleware';

export const useNexStore = create(persist(
(set, get) => ({
// API Config
klingApiKey: '',
setKlingApiKey: (key) => set({ klingApiKey: key }),

// Generación activa
isGenerating: false,
currentJob: null,
progress: 0,
setGenerating: (val) => set({ isGenerating: val }),
setCurrentJob: (job) => set({ currentJob: job }),
setProgress: (p) => set({ progress: p }),

// Historial
history: [],
addToHistory: (item) => set(s => ({ history: [item, ...s.history].slice(0, 100) })),
clearHistory: () => set({ history: [] }),

// Studio activo
activeStudio: 'video',
setActiveStudio: (studio) => set({ activeStudio: studio }),

// Proyectos
projects: [],
addProject: (p) => set(s => ({ projects: [p, ...s.projects] })),

// UI
sidebarOpen: true,
toggleSidebar: () => set(s => ({ sidebarOpen: !s.sidebarOpen })),
}),
{ name: 'nexframe-storage', partialize: (s) => ({ klingApiKey: s.klingApiKey, history:
s.history, projects: s.projects }) }
));
```

SECCIÓN 06

El dashboard es el corazón de NEXFRAME FILMS. Layout: Sidebar izquierdo fijo + área principal. Inspiración visual: Higgsfield pero más dark, más cinematic, con la paleta roja/negra/dorada de YANKYFILMS.

LAYOUT PRINCIPAL

```
// src/components/layout/NexFrameShell.jsx
import { useState } from 'react';
import { Sidebar } from './Sidebar';
import { TopBar } from './TopBar';
import { useNexStore } from '@/lib/store';

export function NexFrameShell({ children }) {
const { sidebarOpen, toggleSidebar } = useNexStore();
return (
<div className="flex h-screen bg-nf-black overflow-hidden">
{/* Sidebar */}
<Sidebar open={sidebarOpen} />
{/* Main */}
<div className={`flex-1 flex flex-col transition-all duration-300 ${sidebarOpen ? "ml-64" : "ml-16"}`}>
<TopBar onMenuClick={toggleSidebar} />
<main className="flex-1 overflow-y-auto p-6 bg-nf-deep">
{children}
</main>
</div>
</div>
);
}
```

SIDEBAR — NAVEGACIÓN COMPLETA

```
// src/components/layout/Sidebar.jsx
import { Film, Image, Music, Zap, Mic2, Video, FileText, Clapperboard,
Settings, BarChart2, FolderOpen, ChevronLeft } from 'lucide-react';
import Link from 'next/link';
import { usePathname } from 'next/navigation';
import { cn } from '@/lib/utils';
```

```
const STUDIOS = [
{ href: '/studio/video', icon: Film, label: 'Video Studio', badge: null },
{ href: '/studio/image', icon: Image, label: 'Image Studio', badge: null },
{ href: '/studio/sound', icon: Music, label: 'Sound Studio', badge: null },
{ href: '/studio/effects', icon: Zap, label: 'Effects Studio', badge: null },
{ href: '/studio/lipsync', icon: Mic2, label: 'Lip Sync Studio', badge: null },
{ href: '/studio/cinema', icon: Clapperboard,label: 'Cinema Studio', badge: null },
{ href: '/studio/documentary', icon: Video, label: 'Documentary', badge: 'NEW' },
{ href: '/studio/musicvideo', icon: Film, label: 'Music Video', badge: 'NEW' },
{ href: '/studio/script', icon: FileText, label: 'Script Engine', badge: 'NEW' },
];

const BOTTOM_LINKS = [
{ href: '/projects', icon: FolderOpen, label: 'Mis Proyectos' },
{ href: '/history', icon: BarChart2, label: 'Historial' },
{ href: '/settings', icon: Settings, label: 'Configuración' },
];

export function Sidebar({ open }) {
const path = usePathname();
return (
<aside className={`fixed left-0 top-0 h-full bg-nf-dark border-r border-nf-border
flex flex-col transition-all duration-300 z-50 ${open ? "w-64" : "w-16"}`}>
{/* LOGO */}
<div className="h-16 flex items-center px-4 border-b border-nf-border shrink-0">
<div className="flex items-center gap-3">
<div className="w-8 h-8 bg-nf-red rounded flex items-center justify-center shrink-0">
<span className="text-white font-heading text-lg">N</span>
</div>
{open && (
<div>
<p className="font-heading text-lg text-white leading-none tracking-wide">NEXFRAME</p>
<p className="text-[10px] text-nf-red font-body">The Future of Filmmaking</p>
</div>
)}
</div>
</div>
{/* STUDIOS NAV */}
<nav className="flex-1 px-2 py-4 space-y-1 overflow-y-auto">
{open && <p className="text-[10px] text-nf-grey-dim uppercase tracking-widest px-2
mb-2">Studios</p>}
{STUDIOS.map(({ href, icon: Icon, label, badge }) => (
<Link key={href} href={href}
className={cn(
```

```
'flex items-center gap-3 px-2 py-2 rounded text-sm transition-all group relative',
path === href
? 'bg-nf-red/10 text-nf-red border-l-2 border-nf-red'
: 'text-nf-grey hover:text-nf-white hover:bg-nf-surface'
)}>
<Icon size={18} className="shrink-0" />
{open && (
<span className="flex-1">{label}</span>
)}
{open && badge && (
<span className="text-[9px] bg-nf-red text-white px-1.5 py-0.5 rounded
font-bold">{badge}</span>
)}
</Link>
))}
</nav>
{/* BOTTOM */}
<div className="px-2 py-4 border-t border-nf-border space-y-1">
{BOTTOM_LINKS.map(({ href, icon: Icon, label }) => (
<Link key={href} href={href}
className="flex items-center gap-3 px-2 py-2 rounded text-sm text-nf-grey
hover:text-nf-white hover:bg-nf-surface transition-all">
<Icon size={18} className="shrink-0" />
{open && <span>{label}</span>}
</Link>
))}
</div>
</aside>
);
}
```

TOPBAR

```
// src/components/layout/TopBar.jsx
import { Menu, Bell, Key, Cpu } from 'lucide-react';
import { useNexStore } from '@/lib/store';
import { ApiKeyModal } from '@/components/ui/ApiKeyModal';
import { useState } from 'react';

export function TopBar({ onMenuClick }) {
const { klingApiKey, isGenerating } = useNexStore();
const [showKey, setShowKey] = useState(false);
return (
```

```
<header className="h-16 bg-nf-dark border-b border-nf-border flex items-center justify-between px-6 shrink-0">
<div className="flex items-center gap-4">
<button onClick={onMenuClick} className="text-nf-grey hover:text-white transition-colors">
<Menu size={20} />
</button>
{isGenerating && (
<div className="flex items-center gap-2 text-nf-amber text-sm animate-pulse">
<Cpu size={16} />
<span>Generando...</span>
</div>
)}
</div>
<div className="flex items-center gap-3">
<div className={`flex items-center gap-2 text-xs px-3 py-1.5 rounded border ${klingApiKey ? "border-nf-green text-nf-green" : "border-nf-red text-nf-red"}`}>
<div className={`w-1.5 h-1.5 rounded-full ${klingApiKey ? "bg-nf-green" : "bg-nf-red"} animate-pulse`} />
{klingApiKey ? "API Conectada" : "Sin API Key"}
</div>
<button onClick={() => setShowKey(true)} className="p-2 text-nf-grey hover:text-white hover:bg-nf-surface rounded transition-all">
<Key size={18} />
</button>
</div>
{showKey && <ApiKeyModal onClose={() => setShowKey(false)} />}
</header>
);
}
```

SECCIÓN 07

El Video Studio soporta Text-to-Video e Image-to-Video. Auto-detecta el modo según si hay imagen subida. Usa la lógica del VideoStudio.jsx original pero con klingClient como backend.

VIDEO STUDIO COMPLETO

```
// src/components/studios/VideoStudioNF.jsx
import { useState, useCallback } from 'react';
import { useDropzone } from 'react-dropzone';
import { generateTextToVideo, generateImageToVideo, uploadFile } from
'@/lib/kling-client';
import { useNexStore } from '@/lib/store';
import { Button } from '@/components/ui/Button';
import { Select } from '@/components/ui/Select';
import { Slider } from '@/components/ui/Slider';
import toast from 'react-hot-toast';
import { Film, Upload, X, Download, Loader2 } from 'lucide-react';

const MODELS = [
{ value: 'kling-v1', label: 'Kling v1 — Standard' },
{ value: 'kling-v1-5', label: 'Kling v1.5 — HD' },
{ value: 'kling-v1-6', label: 'Kling v1.6 — HD Pro' },
{ value: 'kling-v2-master', label: 'Kling v2 Master' },
{ value: 'kling-v2-1-master', label: 'Kling v2.1 Master — Best' },
{ value: 'kling-v3', label: 'Kling v3 Omni — Latest' },
];

const RATIOS = ["16:9","9:16","1:1","4:3","3:4"];
const DURATIONS = ["5","10"];
const MODES = [{value:"std",label:"Standard"},{value:"pro",label:"Professional"}];

export function VideoStudioNF() {
const [prompt, setPrompt] = useState("");
const [negPrompt, setNegPrompt] = useState("");
const [model, setModel] = useState("kling-v2-1-master");
const [ratio, setRatio] = useState("16:9");
const [duration, setDuration] = useState("5");
const [mode, setMode] = useState("std");
const [cfgScale, setCfgScale] = useState(0.5);
const [imageFile, setImageFile] = useState(null);
const [imageUrl, setImageUrl] = useState("");
const [imagePreview, setImagePreview] = useState("");
```

```
const [result, setResult] = useState(null);
const [loading, setLoading] = useState(false);
const { addToHistory, setGenerating } = useNexStore();

const onDrop = useCallback(async (files) => {
const file = files[0];
if (!file) return;
setImageFile(file);
setImagePreview(URL.createObjectURL(file));
toast.loading('Subiendo imagen...');
const url = await uploadFile(file);
setImageUrl(url);
toast.dismiss(); toast.success('Imagen lista');
}, []);

const { getRootProps, getInputProps, isDragActive } = useDropzone({
onDrop, accept: {"image/*":[]}, maxFiles: 1
});

const handleGenerate = async () => {
if (!prompt.trim() && !imageUrl) {
toast.error('Escribe un prompt o sube una imagen');
return;
}
setLoading(true);
setGenerating(true);
setResult(null);
try {
const params = { model, prompt, negative_prompt: negPrompt,
ratio, duration, mode, cfg_scale: cfgScale };
const data = imageUrl
? await generateImageToVideo({ ...params, image_url: imageUrl })
: await generateTextToVideo(params);
const videoUrl = data?.works?.[0]?.video?.resource
|| data?.task_result?.videos?.[0]?.url
|| data?.videos?.[0]?.url;
setResult(videoUrl);
addToHistory({ type:"video", url:videoUrl, prompt, model, created:Date.now() });
toast.success('¡Video generado!');
} catch (e) {
toast.error(e.message || 'Error al generar');
} finally {
setLoading(false);
setGenerating(false);
}
```

```
};

return (
<div className="grid grid-cols-1 lg:grid-cols-2 gap-6 animate-slide-up">
{/* LEFT — Controls */}
<div className="space-y-4">
<div className="flex items-center gap-3 mb-6">
<Film className="text-nf-red" size={22} />
<h2 className="font-heading text-2xl text-white tracking-wide">
{imageUrl ? "IMAGE TO VIDEO" : "TEXT TO VIDEO"}
</h2>
</div>
{/* Image Upload */}
<div {...getRootProps()} className={`border-2 border-dashed rounded-lg p-4 cursor-pointer
transition-all
${isDragActive ? "border-nf-red bg-nf-red/5" : "border-nf-border
hover:border-nf-red/50"}`}>
<input {...getInputProps()} />
{imagePreview ? (
<div className="relative">
<img src={imagePreview} className="w-full h-32 object-cover rounded" />
<button onClick={(e)=>{e.stopPropagation();setImageFile(null);setImageUrl("");setImagePrev
iew("");}}
className="absolute top-2 right-2 bg-black/60 rounded-full p-1 text-white">
<X size={14} />
</button>
</div>
) : (
<div className="text-center py-4">
<Upload className="mx-auto text-nf-grey mb-2" size={24} />
<p className="text-nf-grey text-sm">Sube imagen de inicio (opcional)</p>
</div>
)}
</div>
{/* Prompt */}
<textarea value={prompt} onChange={e=>setPrompt(e.target.value)}
placeholder="Describe tu visión cinematográfica..."
rows={4}
className="w-full bg-nf-surface border border-nf-border rounded-lg px-4 py-3
text-white placeholder-nf-grey text-sm resize-none focus:outline-none
focus:border-nf-red transition-colors" />
<textarea value={negPrompt} onChange={e=>setNegPrompt(e.target.value)}
placeholder="Prompt negativo (opcional)..."
```

```
rows={2}
className="w-full bg-nf-surface border border-nf-border rounded-lg px-4 py-3
text-nf-grey placeholder-nf-grey-dim text-sm resize-none focus:outline-none
focus:border-nf-border transition-colors" />
{/* Settings Grid */}
<div className="grid grid-cols-2 gap-3">
<div>
<label className="text-xs text-nf-grey mb-1 block">MODELO</label>
<select value={model} onChange={e=>setModel(e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-3 py-2 text-sm
text-white">
{MODELS.map(m=><option key={m.value} value={m.value}>{m.label}</option>)}
</select>
</div>
<div>
<label className="text-xs text-nf-grey mb-1 block">RATIO</label>
<select value={ratio} onChange={e=>setRatio(e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-3 py-2 text-sm
text-white">
{RATIOS.map(r=><option key={r} value={r}>{r}</option>)}
</select>
</div>
<div>
<label className="text-xs text-nf-grey mb-1 block">DURACIÓN</label>
<select value={duration} onChange={e=>setDuration(e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-3 py-2 text-sm
text-white">
{DURATIONS.map(d=><option key={d} value={d}>{d}s</option>)}
</select>
</div>
<div>
<label className="text-xs text-nf-grey mb-1 block">MODO</label>
<select value={mode} onChange={e=>setMode(e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-3 py-2 text-sm
text-white">
{MODES.map(m=><option key={m.value} value={m.value}>{m.label}</option>)}
</select>
</div>
</div>
{/* CFG Scale */}
<div>
<label className="text-xs text-nf-grey mb-2 block">CFG SCALE: {cfgScale}</label>
<input type="range" min={0} max={1} step={0.05} value={cfgScale}
```

```
onChange={e=>setCfgScale(parseFloat(e.target.value))}
className="w-full accent-nf-red" />
</div>
<button onClick={handleGenerate} disabled={loading}
className="w-full h-12 bg-nf-red hover:bg-red-600 disabled:opacity-50 text-white
font-heading text-lg tracking-widest rounded-lg transition-all
shadow-lg shadow-nf-red/20 hover:shadow-nf-red/40 flex items-center justify-center gap-2">
{loading ? <><Loader2 size={18} className="animate-spin" /> GENERANDO...</> : "GENERAR
VIDEO"}
</button>
</div>
{/* RIGHT — Result */}
<div className="bg-nf-card border border-nf-border rounded-xl overflow-hidden">
{result ? (
<div className="h-full flex flex-col">
<video src={result} controls autoPlay loop
className="flex-1 w-full object-contain bg-black" />
<div className="p-4 border-t border-nf-border flex gap-2">
<a href={result} download className="flex-1">
<button className="w-full h-10 border border-nf-green text-nf-green rounded
hover:bg-nf-green hover:text-black transition-all flex items-center justify-center gap-2
text-sm">
<Download size={16} /> Descargar
</button>
</a>
</div>
</div>
) : (
<div className="h-full flex flex-col items-center justify-center text-nf-grey p-8">
<Film size={48} className="mb-4 opacity-20" />
<p className="text-lg font-heading tracking-wide">TU VIDEO APARECERÁ AQUÍ</p>
<p className="text-sm mt-2 text-center">Configura el prompt y pulsa Generar</p>
</div>
)}
</div>
</div>
);
}
```

SECCIÓN 08

```
// src/components/studios/ImageStudioNF.jsx
import { useState, useCallback } from 'react';
import { generateImage, uploadFile } from '@/lib/kling-client';
import { useNexStore } from '@/lib/store';
import { useDropzone } from 'react-dropzone';
import toast from 'react-hot-toast';
import { Image as ImageIcon, Download, Loader2, Upload, X } from 'lucide-react';

const IMAGE_MODELS = ['kolors-v1'];
const RATIOS = ["1:1","16:9","9:16","4:3","3:4","3:2","2:3"];

export function ImageStudioNF() {
const [prompt, setPrompt] = useState("");
const [negPrompt, setNegPrompt] = useState("");
const [model, setModel] = useState("kolors-v1");
const [ratio, setRatio] = useState("1:1");
const [n, setN] = useState(1);
const [refImage, setRefImage] = useState("");
const [fidelity, setFidelity] = useState(0.5);
const [results, setResults] = useState([]);
const [loading, setLoading] = useState(false);
const { addToHistory, setGenerating } = useNexStore();

const onDrop = useCallback(async (files) => {
const url = await uploadFile(files[0]);
setRefImage(url);
toast.success('Imagen de referencia cargada');
}, []);
const { getRootProps, getInputProps } = useDropzone({ onDrop, accept:{"image/*":[]},
maxFiles:1 });

const handleGenerate = async () => {
if (!prompt.trim()) { toast.error('Escribe un prompt'); return; }
setLoading(true); setGenerating(true); setResults([]);
try {
const data = await generateImage({
model, prompt, negative_prompt: negPrompt,
aspect_ratio: ratio, n,
image_reference: refImage || undefined,
image_fidelity: refImage ? fidelity : undefined,
});
```

```
const images = data?.task_result?.images || data?.images || [];
const urls = images.map(i => i.url || i.resource);
setResults(urls);
urls.forEach(url => addToHistory({ type:"image", url, prompt, model, created:Date.now()
}));
toast.success(`${urls.length} imagen(es) generada(s)`);
} catch(e) {
toast.error(e.message || 'Error');
} finally {
setLoading(false); setGenerating(false);
}
};

return (
<div className="grid grid-cols-1 lg:grid-cols-2 gap-6 animate-slide-up">
<div className="space-y-4">
<div className="flex items-center gap-3 mb-4">
<ImageIcon className="text-nf-red" size={22} />
<h2 className="font-heading text-2xl text-white tracking-wide">IMAGE STUDIO</h2>
</div>
<textarea value={prompt} onChange={e=>setPrompt(e.target.value)}
placeholder="Describe tu imagen cinematográfica..."
rows={5}
className="w-full bg-nf-surface border border-nf-border rounded-lg px-4 py-3
text-white placeholder-nf-grey text-sm resize-none focus:outline-none focus:border-nf-red
transition-colors"/>
<div {...getRootProps()} className="border border-dashed border-nf-border rounded-lg p-3
cursor-pointer hover:border-nf-red/50">
<input {...getInputProps()} />
<div className="flex items-center gap-2 text-nf-grey text-sm">
<Upload size={16}/> Imagen de referencia (opcional)
</div>
{refImage && <p className="text-nf-green text-xs mt-1">✓ Referencia cargada</p>}
</div>
<div className="grid grid-cols-3 gap-3">
<div><label className="text-xs text-nf-grey mb-1 block">RATIO</label>
<select value={ratio} onChange={e=>setRatio(e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-2 py-2 text-sm
text-white">
{RATIOS.map(r=><option key={r}>{r}</option>)}</select></div>
<div><label className="text-xs text-nf-grey mb-1 block">CANTIDAD</label>
<select value={n} onChange={e=>setN(+e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-2 py-2 text-sm
text-white">
```

```
{[1,2,4].map(v=><option key={v}>{v}</option>)}</select></div>
<div><label className="text-xs text-nf-grey mb-1 block">MODELO</label>
<select value={model} onChange={e=>setModel(e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-2 py-2 text-sm
text-white">
{IMAGE_MODELS.map(m=><option key={m}>{m}</option>)}</select></div>
</div>
<button onClick={handleGenerate} disabled={loading}
className="w-full h-12 bg-nf-red hover:bg-red-600 disabled:opacity-50 text-white
font-heading text-lg tracking-widest rounded-lg transition-all flex items-center
justify-center gap-2">
{loading?<><Loader2 size={18} className="animate-spin"/>GENERANDO...</>:"GENERAR IMAGEN"}
</button>
</div>
<div className={`grid ${results.length>1?"grid-cols-2":"grid-cols-1"} gap-3 bg-nf-card
border border-nf-border rounded-xl p-4`}>
{results.length>0 ? results.map((url,i) => (
<div key={i} className="relative group">
<img src={url} className="w-full rounded-lg object-cover"/>
<a href={url} download className="absolute inset-0 flex items-center justify-center
bg-black/60 opacity-0 group-hover:opacity-100 transition-opacity rounded-lg">
<Download className="text-white" size={28}/>
</a>
</div>
)) : (
<div className="flex flex-col items-center justify-center h-64 text-nf-grey">
<ImageIcon size={48} className="mb-3 opacity-20"/>
<p className="font-heading tracking-wide">TU IMAGEN AQUÍ</p>
</div>
)}
</div>
</div>
);
}
```

SECCIÓN 09

```
// src/components/studios/SoundStudioNF.jsx
import { useState } from 'react';
```

```
import { generateSound } from '@/lib/kling-client';
import { useNexStore } from '@/lib/store';
import toast from 'react-hot-toast';
import { Music, Loader2, Play, Download, Volume2 } from 'lucide-react';

export function SoundStudioNF() {
const [prompt, setPrompt] = useState("");
const [negPrompt, setNegPrompt] = useState("");
const [duration, setDuration] = useState(5);
const [result, setResult] = useState(null);
const [loading, setLoading] = useState(false);
const { addToHistory, setGenerating } = useNexStore();

const handleGenerate = async () => {
if (!prompt.trim()) { toast.error('Describe el audio a generar'); return; }
setLoading(true); setGenerating(true);
try {
const data = await generateSound({ prompt, negative_prompt: negPrompt, duration });
const url = data?.task_result?.audios?.[0]?.url || data?.audios?.[0]?.url;
setResult(url);
addToHistory({ type:"audio", url, prompt, created:Date.now() });
toast.success('¡Audio generado!');
} catch(e) {
toast.error(e.message || 'Error');
} finally { setLoading(false); setGenerating(false); }
};

return (
<div className="max-w-2xl mx-auto space-y-6 animate-slide-up">
<div className="flex items-center gap-3 mb-4">
<Music className="text-nf-red" size={22} />
<h2 className="font-heading text-2xl text-white tracking-wide">SOUND STUDIO</h2>
</div>
<textarea value={prompt} onChange={e=>setPrompt(e.target.value)}
placeholder="Describe el sonido o música a generar... ej: dramatic cinematic orchestra,
dark ambient..."
rows={5}
className="w-full bg-nf-surface border border-nf-border rounded-lg px-4 py-3
text-white placeholder-nf-grey text-sm resize-none focus:outline-none
focus:border-nf-red"/>
<div className="flex items-center gap-4">
<label className="text-xs text-nf-grey whitespace-nowrap">DURACIÓN: {duration}s</label>
<input type="range" min={1} max={10} value={duration}
onChange={e=>setDuration(+e.target.value)}
className="flex-1 accent-nf-red"/>
</div>
```

```
<button onClick={handleGenerate} disabled={loading}
className="w-full h-12 bg-nf-red hover:bg-red-600 disabled:opacity-50 text-white
font-heading text-lg tracking-widest rounded-lg transition-all flex items-center
justify-center gap-2">
{loading?<><Loader2 size={18} className="animate-spin"/>GENERANDO...</>:"GENERAR AUDIO"}
</button>
{result && (
<div className="bg-nf-card border border-nf-border rounded-xl p-6">
<div className="flex items-center gap-3 mb-4">
<Volume2 className="text-nf-green" size={20}/>
<span className="text-nf-green text-sm font-semibold">Audio generado</span>
</div>
<audio controls src={result} className="w-full"/>
<a href={result} download className="mt-3 flex items-center gap-2 text-nf-grey
hover:text-white text-sm transition-colors">
<Download size={16}/> Descargar audio
</a>
</div>
)}
</div>
);
}
```

SECCIÓN 11

```
// src/components/studios/LipSyncStudioNF.jsx
import { useState, useCallback } from 'react';
import { generateLipSync, uploadFile } from '@/lib/kling-client';
import { useDropzone } from 'react-dropzone';
import { useNexStore } from '@/lib/store';
import toast from 'react-hot-toast';
import { Mic2, Upload, Loader2, Download } from 'lucide-react';

type Mode = 'image' | 'video';

export function LipSyncStudioNF() {
const [mode, setMode] = useState<Mode>('image');
const [mediaUrl, setMediaUrl] = useState("");
const [audioUrl, setAudioUrl] = useState("");
const [result, setResult] = useState(null);
const [loading, setLoading] = useState(false);
const [mediaPreview, setMediaPreview] = useState("");
const { addToHistory, setGenerating } = useNexStore();

const onDropMedia = useCallback(async (files) => {
const file = files[0];
setMediaPreview(URL.createObjectURL(file));
toast.loading('Subiendo media...');
const url = await uploadFile(file);
setMediaUrl(url);
toast.dismiss(); toast.success('Media lista');
}, []);

const onDropAudio = useCallback(async (files) => {
toast.loading('Subiendo audio...');
const url = await uploadFile(files[0]);
setAudioUrl(url);
toast.dismiss(); toast.success('Audio listo');
}, []);

const { getRootProps: getMediaProps, getInputProps: getMediaInput } = useDropzone({
onDrop: onDropMedia, accept: mode==='image' ? {'image/*':[]} : {'video/*':[]}, maxFiles:1
});
const { getRootProps: getAudioProps, getInputProps: getAudioInput } = useDropzone({
onDrop: onDropAudio, accept: {'audio/*':[]}, maxFiles:1
});
```

```
const handleGenerate = async () => {
if (!mediaUrl || !audioUrl) { toast.error('Sube media y audio primero'); return; }
setLoading(true); setGenerating(true);
try {
const params = {
[mode === 'image' ? 'image_url' : 'video_url']: mediaUrl,
audio_url: audioUrl,
};
const data = await generateLipSync(params);
const url = data?.task_result?.videos?.[0]?.url || data?.videos?.[0]?.url;
setResult(url);
addToHistory({ type:"lipsync", url, created:Date.now() });
toast.success('¡Lip sync completado!');
} catch(e) {
toast.error(e.message || 'Error');
} finally { setLoading(false); setGenerating(false); }
};

return (
<div className="max-w-2xl mx-auto space-y-6 animate-slide-up">
<div className="flex items-center gap-3 mb-4">
<Mic2 className="text-nf-red" size={22}/>
<h2 className="font-heading text-2xl text-white tracking-wide">LIP SYNC STUDIO</h2>
</div>
{/* Mode Toggle */}
<div className="flex gap-2">
{(['image','video'] as Mode[]).map(m=>(
<button key={m} onClick={()=>{setMode(m);setMediaUrl("");setMediaPreview("");}}
className={`px-4 py-2 rounded text-sm font-semibold transition-all ${mode===m
?"bg-nf-red text-white":"border border-nf-border text-nf-grey hover:text-white"}`}>
{m === "image" ? "FOTO + AUDIO" : "VIDEO + AUDIO"}
</button>
))}
</div>
{/* Media Upload */}
<div {...getMediaProps()} className="border-2 border-dashed border-nf-border rounded-xl
p-6
cursor-pointer hover:border-nf-red/50 transition-all">
<input {...getMediaInput()}/>
{mediaPreview ? (
mode==="image"
? <img src={mediaPreview} className="w-full h-40 object-cover rounded"/>
: <video src={mediaPreview} className="w-full h-40 object-cover rounded" controls/>
```

```
) : (
<div className="text-center">
<Upload className="mx-auto text-nf-grey mb-2" size={32}/>
<p className="text-nf-grey text-sm">Sube tu {mode === "image" ? "foto/retrato" :
"video"}</p>
</div>
)}
</div>
{/* Audio Upload */}
<div {...getAudioProps()} className="border-2 border-dashed border-nf-border rounded-xl
p-4
cursor-pointer hover:border-nf-amber/50 transition-all">
<input {...getAudioInput()}/>
<div className="flex items-center gap-3 text-nf-grey">
<Upload size={20}/>
<span className="text-sm">{audioUrl ? "✓ Audio cargado" : "Sube el archivo de audio (.mp3,
.wav, .m4a)"}</span>
</div>
</div>
<button onClick={handleGenerate} disabled={loading}
className="w-full h-12 bg-nf-red hover:bg-red-600 disabled:opacity-50 text-white
font-heading text-lg tracking-widest rounded-lg transition-all flex items-center
justify-center gap-2">
{loading?<><Loader2 size={18} className="animate-spin"/>PROCESANDO...</>:"GENERAR LIP
SYNC"}
</button>
{result && (
<div className="bg-nf-card border border-nf-green rounded-xl overflow-hidden">
<video src={result} controls autoPlay loop className="w-full"/>
<div className="p-3"><a href={result} download>
<button className="flex items-center gap-2 text-nf-green text-sm hover:text-white">
<Download size={16}/> Descargar video</button></a></div>
</div>
)}
</div>
);
}
```

SECCIÓN 12

Studio exclusivo para producción de documentales. Flujo: Tema → Script IA → Narración → Prompts de video → Generación → Proyecto. Integra el Script Engine con Video Studio.

```
// src/components/studios/DocumentaryStudioNF.jsx
import { useState } from 'react';
import { generateTextToVideo } from '@/lib/kling-client';
import { useNexStore } from '@/lib/store';
import toast from 'react-hot-toast';
import { Video, FileText, Film, Loader2, ChevronRight, Check } from 'lucide-react';

const STEPS = ["TEMA", "GUION", "ESCENAS", "GENERACIÓN", "PROYECTO"];

export function DocumentaryStudioNF() {
const [step, setStep] = useState(0);
const [topic, setTopic] = useState("");
const [style, setStyle] = useState("investigativo");
const [duration, setDuration] = useState("35-40");
const [script, setScript] = useState("");
const [scenes, setScenes] = useState([]);
const [videos, setVideos] = useState([]);
const [loading, setLoading] = useState(false);
const { klingApiKey } = useNexStore();

// Generar guion con IA (llamada a Next.js API route)
const generateScript = async () => {
if (!topic.trim()) { toast.error('Define el tema'); return; }
setLoading(true);
try {
const res = await fetch("/api/script/generate", {
method: 'POST',
headers: {'Content-Type':'application/json'},
body: JSON.stringify({ topic, style, duration, type:"documentary" }),
});
const data = await res.json();
setScript(data.script);
setStep(1);
toast.success('Guion generado');
} catch(e) {
toast.error('Error generando guion');
} finally { setLoading(false); }
};
```

```
// Extraer escenas del guion
const generateScenes = async () => {
setLoading(true);
try {
const res = await fetch("/api/script/scenes", {
method: 'POST',
headers: {'Content-Type':'application/json'},
body: JSON.stringify({ script }),
});
const data = await res.json();
setScenes(data.scenes); // Array de {prompt, duration, description}
setStep(2);
toast.success(`${data.scenes.length} escenas extraídas`);
} finally { setLoading(false); }
};

// Generar todos los videos de las escenas
const generateAllVideos = async () => {
setLoading(true);
toast.loading('Generando escenas con Kling AI...');
const results = [];
for (const scene of scenes) {
try {
const data = await generateTextToVideo({
model: 'kling-v2-1-master',
prompt: scene.prompt,
duration: scene.duration || "5",
aspect_ratio: "16:9",
mode: 'pro',
});
const url = data?.works?.[0]?.video?.resource || data?.videos?.[0]?.url;
results.push({ ...scene, videoUrl: url });
} catch(e) {
results.push({ ...scene, videoUrl: null, error: e.message });
}
}
setVideos(results);
setStep(3);
toast.dismiss(); toast.success('Escenas generadas');
setLoading(false);
};

return (
<div className="space-y-6 animate-slide-up">
```

```
{/* Header */}
<div className="flex items-center gap-3 mb-4">
<Video className="text-nf-red" size={22}/>
<h2 className="font-heading text-2xl text-white tracking-wide">DOCUMENTARY STUDIO</h2>
</div>
{/* Stepper */}
<div className="flex items-center gap-2">
{STEPS.map((s,i) => (
<div key={s} className="flex items-center gap-2">
<div className={`flex items-center gap-1.5 px-3 py-1.5 rounded text-xs font-bold
transition-all ${
i < step ? "bg-nf-green/20 text-nf-green" :
i === step ? "bg-nf-red text-white" :
"bg-nf-surface text-nf-grey"
}`}>
{i < step ? <Check size={12}/> : null}
{s}
</div>
{i < STEPS.length-1 && <ChevronRight size={14} className="text-nf-border"/>}
</div>
))}
</div>
{/* STEP 0: Tema */}
{step === 0 && (
<div className="space-y-4 max-w-xl">
<div>
<label className="text-xs text-nf-grey mb-1 block">TEMA DEL DOCUMENTAL</label>
<input value={topic} onChange={e=>setTopic(e.target.value)}
placeholder="ej: La Reserva Federal — Historia clasificada"
className="w-full bg-nf-surface border border-nf-border rounded px-4 py-3 text-white
placeholder-nf-grey focus:border-nf-red outline-none"/>
</div>
<div className="grid grid-cols-2 gap-3">
<div>
<label className="text-xs text-nf-grey mb-1 block">ESTILO</label>
<select value={style} onChange={e=>setStyle(e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-3 py-2 text-sm
text-white">
<option value="investigativo">Investigativo</option>
<option value="conspirativo">Conspirativo</option>
<option value="historico">Histórico</option>
<option value="cientifico">Científico</option>
<option value="codigo-blanco">Código Blanco</option>
```

```
</select>
</div>
<div>
<label className="text-xs text-nf-grey mb-1 block">DURACIÓN (MIN)</label>
<select value={duration} onChange={e=>setDuration(e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-3 py-2 text-sm
text-white">
<option value="20-25">20-25 min</option>
<option value="35-40">35-40 min</option>
<option value="45-60">45-60 min</option>
</select>
</div>
</div>
<button onClick={generateScript} disabled={loading}
className="w-full h-12 bg-nf-red hover:bg-red-600 disabled:opacity-50 text-white
font-heading text-lg tracking-widest rounded-lg transition-all flex items-center
justify-center gap-2">
{loading?<><Loader2 size={18} className="animate-spin"/>GENERANDO GUION...</>:"CREAR
GUION"}
</button>
</div>
)}
{/* STEP 1: Guion */}
{step === 1 && (
<div className="space-y-4">
<label className="text-xs text-nf-grey">GUION GENERADO — Puedes editarlo</label>
<textarea value={script} onChange={e=>setScript(e.target.value)} rows={20}
className="w-full bg-nf-surface border border-nf-border rounded-lg px-4 py-3
text-white text-sm font-mono resize-none focus:outline-none focus:border-nf-red"/>
<button onClick={generateScenes} disabled={loading}
className="w-full h-12 bg-nf-red hover:bg-red-600 disabled:opacity-50 text-white
font-heading text-lg tracking-widest rounded-lg flex items-center justify-center gap-2">
{loading?<><Loader2 size={18} className="animate-spin"/>EXTRAYENDO...</>:"EXTRAER
ESCENAS"}
</button>
</div>
)}
{/* STEP 2: Escenas */}
{step === 2 && (
<div className="space-y-4">
<p className="text-nf-grey text-sm">{scenes.length} escenas listas para generar</p>
<div className="grid grid-cols-1 gap-3 max-h-96 overflow-y-auto pr-2">
{scenes.map((s,i) => (
```

```
<div key={i} className="bg-nf-card border border-nf-border rounded-lg p-4">
<div className="flex items-center gap-2 mb-2">
<span className="text-nf-red text-xs font-bold">ESCENA {i+1}</span>
<span className="text-nf-grey text-xs">{s.duration || "5"}s</span>
</div>
<p className="text-white text-sm">{s.description}</p>
<p className="text-nf-grey-dim text-xs mt-1 font-mono">{s.prompt}</p>
</div>
))}
</div>
<button onClick={generateAllVideos} disabled={loading}
className="w-full h-12 bg-nf-red hover:bg-red-600 disabled:opacity-50 text-white
font-heading text-lg tracking-widest rounded-lg flex items-center justify-center gap-2">
{loading?<><Loader2 size={18} className="animate-spin"/>GENERANDO CON KLING
AI...</>:"GENERAR TODAS LAS ESCENAS"}
</button>
</div>
)}
{/* STEP 3: Videos */}
{step >= 3 && (
<div className="space-y-4">
<p className="text-nf-green text-sm font-semibold">✓
{videos.filter(v=>v.videoUrl).length}/{videos.length} escenas completadas</p>
<div className="grid grid-cols-2 gap-4">
{videos.map((v,i) => (
<div key={i} className="bg-nf-card border border-nf-border rounded-xl overflow-hidden">
<div className="bg-nf-surface px-3 py-2 flex items-center justify-between">
<span className="text-nf-grey text-xs">ESCENA {i+1}</span>
{v.videoUrl ? <span className="text-nf-green text-xs">✓ OK</span>
: <span className="text-nf-red text-xs">✗ ERROR</span>}
</div>
{v.videoUrl && <video src={v.videoUrl} controls loop className="w-full"/>}
</div>
))}
</div>
</div>
)}
</div>
);
}
```

SECCIÓN 13

Flujo completo de music video: Audio → análisis de beats → storyboard → generación de imágenes por escena → generación de video por escena → proyecto final.

```
// src/components/studios/MusicVideoStudioNF.jsx
import { useState, useCallback } from 'react';
import { generateImage, generateImageToVideo, uploadFile } from '@/lib/kling-client';
import { useDropzone } from 'react-dropzone';
import { useNexStore } from '@/lib/store';
import toast from 'react-hot-toast';
import { Music, Film, Upload, Loader2, Wand2, Check, ChevronRight } from 'lucide-react';

const VISUAL_STYLES = [
"cinematic dark, red amber palette, no faces, abstract",
"neon cyberpunk, urban night, faceless silhouettes",
"reggae conscious, nature, rastafarian colors, mystical",
"Latin urban, street art, vibrant colors, no people",
"documentary raw, black and white, gritty texture",
"r&b soul, warm tones, luxurious environments, abstract",
];

export function MusicVideoStudioNF() {
const [audioUrl, setAudioUrl] = useState("");
const [artistName, setArtistName] = useState("");
const [songTitle, setSongTitle] = useState("");
const [genre, setGenre] = useState("reggae-conscious");
const [visualStyle, setVisualStyle] = useState(VISUAL_STYLES[0]);
const [sceneCount, setSceneCount] = useState(8);
const [storyboard, setStoryboard] = useState([]);
const [images, setImages] = useState([]);
const [videos, setVideos] = useState([]);
const [step, setStep] = useState(0);
const [loading, setLoading] = useState(false);
const { addToHistory } = useNexStore();

const onDropAudio = useCallback(async (files) => {
toast.loading('Subiendo audio...');
const url = await uploadFile(files[0]);
setAudioUrl(url);
toast.dismiss(); toast.success('Audio listo');
}, []);
const { getRootProps, getInputProps } = useDropzone({
```

```
onDrop: onDropAudio, accept:{'audio/*':[]}, maxFiles:1
});

// Generar storyboard con IA
const generateStoryboard = async () => {
if (!artistName || !songTitle) { toast.error('Rellena artista y canción'); return; }
setLoading(true);
try {
const res = await fetch("/api/script/musicvideo", {
method:'POST',
headers:{'Content-Type':'application/json'},
body: JSON.stringify({ artistName, songTitle, genre, visualStyle, sceneCount }),
});
const data = await res.json();
setStoryboard(data.scenes);
setStep(1);
toast.success(`Storyboard con ${data.scenes.length} escenas`);
} catch(e) {
toast.error('Error generando storyboard');
} finally { setLoading(false); }
};

// Generar imágenes del storyboard
const generateImages = async () => {
setLoading(true);
const results = [];
for (const scene of storyboard) {
try {
const data = await generateImage({
model: 'kolors-v1',
prompt: `${scene.imagePrompt}, ${visualStyle}`,
negative_prompt: 'faces, people, text, watermark',
aspect_ratio: '16:9',
n: 1,
});
const url = data?.task_result?.images?.[0]?.url;
results.push({ ...scene, imageUrl: url });
} catch(e) {
results.push({ ...scene, imageUrl: null });
}
}
setImages(results);
setStep(2);
setLoading(false);
```

```
toast.success('Imágenes generadas');
};

// Generar videos desde imágenes
const generateVideos = async () => {
setLoading(true);
toast.loading('Animando con Kling AI...');
const results = [];
for (const scene of images) {
if (!scene.imageUrl) { results.push({ ...scene, videoUrl: null }); continue; }
try {
const data = await generateImageToVideo({
model: 'kling-v1-5',
image_url: scene.imageUrl,
prompt: scene.videoPrompt || "",
duration: '5',
aspect_ratio: '16:9',
mode: 'std',
});
const url = data?.works?.[0]?.video?.resource || data?.videos?.[0]?.url;
results.push({ ...scene, videoUrl: url });
} catch(e) {
results.push({ ...scene, videoUrl: null });
}
}
setVideos(results);
setStep(3);
toast.dismiss(); toast.success('Music video generado');
setLoading(false);
};

return (
<div className="space-y-6 animate-slide-up">
<div className="flex items-center gap-3 mb-4">
<Music className="text-nf-red" size={22}/>
<h2 className="font-heading text-2xl text-white tracking-wide">MUSIC VIDEO STUDIO</h2>
</div>
{/* Stepper */}
<div className="flex gap-2 flex-wrap">
{["CONFIGURAR","STORYBOARD","IMÁGENES","VIDEOS"].map((s,i)=>(
<div key={s} className={`px-3 py-1.5 rounded text-xs font-bold ${
i<step?"bg-nf-green/20 text-nf-green":i===step?"bg-nf-red text-white":"bg-nf-surface
text-nf-grey"}`}>
{s}
```

```
</div>
))}
</div>
{/* STEP 0: Config */}
{step === 0 && (
<div className="grid grid-cols-1 md:grid-cols-2 gap-4 max-w-2xl">
<div {...getRootProps()} className="col-span-2 border-2 border-dashed border-nf-border
rounded-xl p-6
cursor-pointer hover:border-nf-red/50 transition-all text-center">
<input {...getInputProps()}/>
<Music className="mx-auto text-nf-grey mb-2" size={28}/>
<p className="text-nf-grey text-sm">{audioUrl?"✓ Audio cargado":"Sube la pista de
audio"}</p>
</div>
<input value={artistName} onChange={e=>setArtistName(e.target.value)}
placeholder="Nombre del artista"
className="bg-nf-surface border border-nf-border rounded px-4 py-3 text-white
placeholder-nf-grey focus:border-nf-red outline-none"/>
<input value={songTitle} onChange={e=>setSongTitle(e.target.value)}
placeholder="Título de la canción"
className="bg-nf-surface border border-nf-border rounded px-4 py-3 text-white
placeholder-nf-grey focus:border-nf-red outline-none"/>
<div>
<label className="text-xs text-nf-grey mb-1 block">GÉNERO</label>
<select value={genre} onChange={e=>setGenre(e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-3 py-2 text-sm
text-white">
<option value="reggae-conscious">Reggae Conscious</option>
<option value="urban-latin">Urban Latin</option>
<option value="rnb">R&B</option>
<option value="hip-hop">Hip Hop</option>
<option value="electronic">Electronic</option>
</select>
</div>
<div>
<label className="text-xs text-nf-grey mb-1 block">ESCENAS</label>
<select value={sceneCount} onChange={e=>setSceneCount(+e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-3 py-2 text-sm
text-white">
{[6,8,10,12,16,20,24,44].map(n=>(
<option key={n} value={n}>{n} escenas</option>
))}
</select>
```

```
</div>
<div className="col-span-2">
<label className="text-xs text-nf-grey mb-1 block">ESTILO VISUAL</label>
<select value={visualStyle} onChange={e=>setVisualStyle(e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-3 py-2 text-sm
text-white">
{VISUAL_STYLES.map(s=>(<option key={s} value={s}>{s.substring(0,60)}...</option>))}
</select>
</div>
<button onClick={generateStoryboard} disabled={loading}
className="col-span-2 h-12 bg-nf-red hover:bg-red-600 disabled:opacity-50 text-white
font-heading text-lg tracking-widest rounded-lg flex items-center justify-center gap-2">
{loading?<><Loader2 size={18} className="animate-spin"/>CREANDO STORYBOARD...</>:"CREAR
STORYBOARD"}
</button>
</div>
)}
{/* STEP 1: Storyboard */}
{step === 1 && (
<div className="space-y-4">
<div className="grid grid-cols-2 md:grid-cols-4 gap-3">
{storyboard.map((scene,i)=>(
<div key={i} className="bg-nf-card border border-nf-border rounded-lg p-3">
<div className="text-nf-red text-xs font-bold mb-1">S{i+1}</div>
<p className="text-white text-xs leading-relaxed">{scene.description}</p>
</div>
))}
</div>
<button onClick={generateImages} disabled={loading}
className="w-full h-12 bg-nf-red hover:bg-red-600 disabled:opacity-50 text-white
font-heading text-lg tracking-widest rounded-lg flex items-center justify-center gap-2">
{loading?<><Loader2 size={18} className="animate-spin"/>GENERANDO IMÁGENES...</>:"GENERAR
IMÁGENES"}
</button>
</div>
)}
{/* STEP 2: Images */}
{step === 2 && (
<div className="space-y-4">
<div className="grid grid-cols-2 md:grid-cols-4 gap-3">
{images.map((scene,i)=>(
<div key={i} className="bg-nf-card border border-nf-border rounded-lg overflow-hidden">
{scene.imageUrl
```

```
? <img src={scene.imageUrl} className="w-full aspect-video object-cover"/>
: <div className="w-full aspect-video bg-nf-surface flex items-center justify-center text-nf-grey text-xs">Error</div>
}
<p className="text-nf-grey text-xs p-2">S{i+1}</p>
</div>
))}
</div>
<button onClick={generateVideos} disabled={loading}
className="w-full h-12 bg-nf-red hover:bg-red-600 disabled:opacity-50 text-white
font-heading text-lg tracking-widest rounded-lg flex items-center justify-center gap-2">
{loading?<><Loader2 size={18} className="animate-spin"/>ANIMANDO CON KLING...</>:"GENERAR VIDEOS"}
</button>
</div>
)}
{/* STEP 3: Videos */}
{step === 3 && (
<div className="space-y-4">
<p className="text-nf-green font-semibold">✓ Music video generado — {videos.filter(v=>v.videoUrl).length}/{videos.length} clips</p>
<div className="grid grid-cols-2 md:grid-cols-3 gap-4">
{videos.map((v,i)=>(
<div key={i} className="bg-nf-card border border-nf-border rounded-xl overflow-hidden">
<div className="bg-nf-surface px-3 py-1.5 text-nf-grey text-xs">Clip {i+1}</div>
{v.videoUrl && <video src={v.videoUrl} controls loop className="w-full"/>}
</div>
))}
</div>
</div>
)}
</div>
);
}
```

SECCIÓN 14

El Cinema Studio usa los controles cinematográficos del repo original pero adaptados a la API de Kling. Construye el prompt a partir de los parámetros de cámara y lo envía a Kling Text-to-Video con modelo pro.

```
// src/components/studios/CinemaStudioNF.jsx
import { useState } from 'react';
import { generateTextToVideo } from '@/lib/kling-client';
import { useNexStore } from '@/lib/store';
import toast from 'react-hot-toast';
import { Clapperboard, Loader2, Download } from 'lucide-react';

const CAMERAS = ["Modular 8K Digital","Full-Frame Cine Digital","Grand Format 70mm Film","Studio Digital S35","Classic 16mm Film","Premium Large Format Digital"];
const LENSES = ["Creative Tilt","Compact Anamorphic","Extreme Macro","70s Cinema Prime","Classic Anamorphic","Premium Modern Prime","Warm Cinema Prime","Vintage Prime","Halation Diffusion"];
const FOCAL_LENGTHS = ["8mm (Ultra-Wide)","14mm","24mm","35mm (Human Eye)","50mm (Portrait)","85mm (Tight Portrait)"];
const APERTURES = ["f/1.4 (Shallow DoF)","f/4 (Balanced)","f/11 (Deep Focus)"];
const MOVEMENTS = ["Static","Pan Left","Pan Right","Tilt Up","Tilt Down","Dolly In","Dolly Out","Crane Up","Crane Down","Orbit","Handheld","Aerial Drone"];

export function CinemaStudioNF() {
const [scene, setScene] = useState("");
const [camera, setCamera] = useState(CAMERAS[0]);
const [lens, setLens] = useState(LENSES[0]);
const [focal, setFocal] = useState(FOCAL_LENGTHS[3]);
const [aperture, setAperture] = useState(APERTURES[0]);
const [movement, setMovement] = useState(MOVEMENTS[0]);
const [duration, setDuration] = useState("5");
const [result, setResult] = useState(null);
const [loading, setLoading] = useState(false);
const { addToHistory, setGenerating } = useNexStore();

const buildCinemaPrompt = () => {
return [
scene,
`Shot on ${camera}`,
`${lens} lens at ${focal}`,
`Aperture ${aperture}`,
`Camera movement: ${movement}`,
"cinematic grade, film grain, professional color grading",
```

```
"dramatic lighting, high production value",
].filter(Boolean).join(", ");
};

const handleGenerate = async () => {
if (!scene.trim()) { toast.error('Describe la escena'); return; }
setLoading(true); setGenerating(true);
try {
const prompt = buildCinemaPrompt();
const data = await generateTextToVideo({
model: 'kling-v2-1-master',
prompt, duration,
aspect_ratio: '16:9',
mode: 'pro',
cfg_scale: 0.7,
});
const url = data?.works?.[0]?.video?.resource || data?.videos?.[0]?.url;
setResult(url);
addToHistory({ type:"cinema", url, prompt, created:Date.now() });
toast.success('Shot cinematográfico generado');
} catch(e) {
toast.error(e.message);
} finally { setLoading(false); setGenerating(false); }
};

return (
<div className="grid grid-cols-1 lg:grid-cols-2 gap-6 animate-slide-up">
<div className="space-y-4">
<div className="flex items-center gap-3 mb-4">
<Clapperboard className="text-nf-red" size={22}/>
<h2 className="font-heading text-2xl text-white tracking-wide">CINEMA STUDIO</h2>
</div>
<textarea value={scene} onChange={e=>setScene(e.target.value)}
placeholder="Describe la escena cinematográfica..."
rows={4}
className="w-full bg-nf-surface border border-nf-border rounded-lg px-4 py-3
text-white placeholder-nf-grey text-sm resize-none focus:border-nf-red outline-none"/>
<div className="grid grid-cols-2 gap-3">
{[["CÁMARA",camera,CAMERAS,setCamera],["LENTE",lens,LENSES,setLens],
["FOCAL",focal,FOCAL_LENGTHS,setFocal],["APERTURA",aperture,APERTURES,setAperture],
["MOVIMIENTO",movement,MOVEMENTS,setMovement],["DURACIÓN",duration,["5","10"],setDuration]
].map(([label,val,opts,setter]) => (
<div key={label}>
<label className="text-xs text-nf-grey mb-1 block">{label}</label>
```

```
<select value={val} onChange={e=>setter(e.target.value)}
className="w-full bg-nf-surface border border-nf-border rounded px-2 py-2 text-xs
text-white">
{opts.map(o=><option key={o} value={o}>{o}</option>)}
</select>
</div>
))}
</div>
<div className="bg-nf-card border border-nf-border rounded-lg p-3">
<p className="text-xs text-nf-grey mb-1">PROMPT GENERADO:</p>
<p className="text-nf-grey-dim text-xs font-mono
leading-relaxed">{buildCinemaPrompt()}</p>
</div>
<button onClick={handleGenerate} disabled={loading}
className="w-full h-12 bg-nf-red hover:bg-red-600 disabled:opacity-50 text-white
font-heading text-lg tracking-widest rounded-lg flex items-center justify-center gap-2">
{loading?<><Loader2 size={18} className="animate-spin"/>RODANDO...</>:"RODAR ESCENA"}
</button>
</div>
<div className="bg-nf-card border border-nf-border rounded-xl overflow-hidden">
{result
? <><video src={result} controls autoPlay loop className="w-full"/>
<div className="p-3"><a href={result} download>
<button className="flex items-center gap-2 text-nf-green text-sm hover:text-white">
<Download size={16}/> Descargar shot</button></a></div></>
: <div className="h-full flex items-center justify-center text-nf-grey">
<Clapperboard size={48} className="opacity-20"/></div>}
</div>
</div>
);
}
```

SECCIÓN 15

API Routes de Next.js que alimentan Documentary Studio y Music Video Studio con generación de guiones, narrativas y storyboards. Usan Claude/GPT como LLM para el texto.

```
// app/api/script/generate/route.js
import { NextResponse } from 'next/server';
```

```
export async function POST(req) {
const { topic, style, duration, type } = await req.json();

const SYSTEM_DOCUMENTARY = `Eres el mejor guionista de documentales cinematográficos en español.
Creas guiones narrativos cinematográficos de alta calidad, sin encabezados de sección,
solo narración continua lista para audio. Mínimo equivalente a ${duration} minutos.`,;

const SYSTEM_MUSICVIDEO = `Eres el mejor director de videos musicales. Creas storyboards
detallados con escenas visuales cinematográficas. Artistas nunca aparecen en cámara.`,;

const systemPrompt = type === "documentary" ? SYSTEM_DOCUMENTARY : SYSTEM_MUSICVIDEO;
const userPrompt = type === "documentary"
? `Crea un guion documental sobre: ${topic}. Estilo: ${style}. Sin encabezados ni notas de producción.`
: `Crea un storyboard de music video para el género ${style} con estos parámetros: ${topic}.`;

// Usar OpenAI o cualquier LLM disponible
const res = await fetch("https://api.openai.com/v1/chat/completions", {
method: 'POST',
headers: {
'Content-Type': 'application/json',
'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`,
},
body: JSON.stringify({
model: 'gpt-4o',
messages: [
{ role: "system", content: systemPrompt },
{ role: "user", content: userPrompt },
],
max_tokens: 4000,
}),
});
const data = await res.json();
const script = data.choices?.[0]?.message?.content || "";
return NextResponse.json({ script });
}

// app/api/script/scenes/route.js — Extrae escenas del guion
export async function POST_SCENES(req) {
const { script } = await req.json();

const res = await fetch("https://api.openai.com/v1/chat/completions", {
method: 'POST',
headers: {
'Content-Type': 'application/json',
'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`,
},
```

```
body: JSON.stringify({
model: 'gpt-4o',
messages: [{
role: "user",
content: `Del siguiente guion, extrae exactamente entre 8 y 16 escenas visuales.
Para cada escena devuelve JSON array con: { description, prompt, duration }.
"prompt" debe ser un prompt cinematográfico en inglés para Kling AI.
"duration" debe ser "5" o "10". Solo JSON, sin markdown.\n\n${script}`,
}],
max_tokens: 3000,
response_format: { type: 'json_object' },
}),
});
const data = await res.json();
const content = JSON.parse(data.choices?.[0]?.message?.content || "{}");
return NextResponse.json({ scenes: content.scenes || [] });
}

// app/api/script/musicvideo/route.js — Genera storyboard de music video
export async function POST_MV(req) {
const { artistName, songTitle, genre, visualStyle, sceneCount } = await req.json();

const res = await fetch("https://api.openai.com/v1/chat/completions", {
method: 'POST',
headers: {
'Content-Type': 'application/json',
'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`,
},
body: JSON.stringify({
model: 'gpt-4o',
messages: [{
role: "user",
content: `Crea un storyboard para video musical: '${songTitle}' de ${artistName}.
Género: ${genre}. Estilo visual: ${visualStyle}.
Genera exactamente ${sceneCount} escenas. Artistas NUNCA aparecen en cámara.
JSON array: [{ description, imagePrompt, videoPrompt }].
imagePrompt: para generar imagen cinematográfica con Kling (en inglés).
videoPrompt: para animar la imagen (en inglés, movimiento de cámara).
Solo JSON sin markdown.`,
}],
max_tokens: 4000,
response_format: { type: 'json_object' },
}),
});
```

```
const data = await res.json();
const content = JSON.parse(data.choices?.[0]?.message?.content || "{}");
return NextResponse.json({ scenes: content.scenes || content || [] });
}
```

SECCIÓN 16

```
// src/components/dashboard/HistoryPanel.jsx
import { useNexStore } from '@/lib/store';
import { Download, Trash2, Film, Image, Music, Mic2, Clapperboard } from 'lucide-react';
import { format } from 'date-fns';

const TYPE_ICONS = {
video: Film, image: Image, audio: Music, lipsync: Mic2, cinema: Clapperboard
};
const TYPE_COLORS = {
video: 'text-nf-red', image: 'text-nf-cyan', audio: 'text-nf-amber',
lipsync: 'text-purple-400', cinema: 'text-nf-gold'
};

export function HistoryPanel() {
const { history, clearHistory } = useNexStore();
if (history.length === 0) return (
<div className="flex flex-col items-center justify-center py-20 text-nf-grey">
<Film size={48} className="mb-4 opacity-20"/>
<p className="font-heading text-xl tracking-wide">SIN GENERACIONES AÚN</p>
<p className="text-sm mt-1">Tus creaciones aparecerán aquí</p>
</div>
);
return (
<div className="space-y-4">
<div className="flex items-center justify-between">
<h2 className="font-heading text-2xl text-white tracking-wide">HISTORIAL</h2>
<button onClick={clearHistory} className="flex items-center gap-2 text-nf-grey
hover:text-nf-red text-sm transition-colors">
<Trash2 size={14}/> Limpiar
</button>
</div>
<div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-4">
{history.map((item,i) => {
const Icon = TYPE_ICONS[item.type] || Film;
const color = TYPE_COLORS[item.type] || "text-nf-grey";
return (
<div key={i} className="bg-nf-card border border-nf-border rounded-xl overflow-hidden
group hover:border-nf-red/50 transition-all">
{item.type === "image" && item.url && (
```

```
<img src={item.url} className="w-full aspect-square object-cover"/>
)}
{(item.type === "video" || item.type === "lipsync" || item.type === "cinema") && item.url
&& (
<video src={item.url} className="w-full aspect-video object-cover" muted loop
onMouseEnter={e=>e.target.play()}
onMouseLeave={e=>{e.target.pause();e.target.currentTime=0;}}/>
)}
{item.type === "audio" && (
<div className="w-full aspect-video bg-nf-surface flex items-center justify-center">
<Music className="text-nf-amber opacity-40" size={40}/>
</div>
)}
<div className="p-3">
<div className="flex items-center justify-between mb-1">
<div className="flex items-center gap-1.5">
<Icon size={12} className={color}/>
<span className={`text-xs font-bold uppercase ${color}`}>{item.type}</span>
</div>
<a href={item.url} download className="opacity-0 group-hover:opacity-100
transition-opacity">
<Download size={14} className="text-nf-grey hover:text-white"/>
</a>
</div>
{item.prompt && <p className="text-nf-grey-dim text-xs truncate">{item.prompt}</p>}
{item.created && <p className="text-nf-grey-dim text-xs mt-1">
{format(new Date(item.created),"dd/MM/yy HH:mm")}</p>}
</div>
</div>
);
})}
</div>
</div>
);
}
```

SECCIÓN 17

```
// src/components/ui/ApiKeyModal.jsx
```

```
import { useState } from 'react';
import { useNexStore } from '@/lib/store';
import { Key, X, Eye, EyeOff, Check, ExternalLink } from 'lucide-react';

export function ApiKeyModal({ onClose }) {
const { klingApiKey, setKlingApiKey } = useNexStore();
const [key, setKey] = useState(klingApiKey);
const [show, setShow] = useState(false);
const [saved, setSaved] = useState(false);

const handleSave = () => {
setKlingApiKey(key.trim());
setSaved(true);
setTimeout(() => { setSaved(false); onClose(); }, 1200);
};

return (
<div className="fixed inset-0 bg-black/80 backdrop-blur-sm flex items-center
justify-center z-[100] p-4">
<div className="bg-nf-card border border-nf-border rounded-2xl w-full max-w-md p-6
shadow-2xl">
<div className="flex items-center justify-between mb-6">
<div className="flex items-center gap-3">
<div className="w-10 h-10 bg-nf-red/10 border border-nf-red/30 rounded-xl
flex items-center justify-center">
<Key className="text-nf-red" size={20}/>
</div>
<div>
<h3 className="font-heading text-lg text-white tracking-wide">API KEY</h3>
<p className="text-xs text-nf-grey">Kling AI — kling.ai</p>
</div>
</div>
<button onClick={onClose} className="text-nf-grey hover:text-white transition-colors">
<X size={20}/>
</button>
</div>
<div className="space-y-4">
<div>
<label className="text-xs text-nf-grey mb-2 block">TU API KEY DE KLING AI</label>
<div className="relative">
<input
type={show ? "text" : "password"}
value={key}
onChange={e=>setKey(e.target.value)}
placeholder="sk-kling-..."
className="w-full bg-nf-surface border border-nf-border rounded-lg px-4 py-3
```

```
text-white placeholder-nf-grey-dim text-sm font-mono pr-12
focus:border-nf-red outline-none transition-colors"/>
<button onClick={()=>setShow(!show)}
className="absolute right-3 top-1/2 -translate-y-1/2 text-nf-grey hover:text-white">
{show ? <EyeOff size={16}/> : <Eye size={16}/>}
</button>
</div>
</div>
<a href="https://kling.ai/dev" target="_blank"
className="flex items-center gap-2 text-nf-cyan text-xs hover:text-cyan-300
transition-colors">
<ExternalLink size={12}/> Obtener API Key en kling.ai/dev
</a>
<button onClick={handleSave}
className={`w-full h-11 rounded-lg font-heading text-base tracking-wider transition-all
flex items-center justify-center gap-2
${saved ? "bg-nf-green text-black" : "bg-nf-red hover:bg-red-600 text-white"}`}>
{saved ? <><Check size={18}/> GUARDADO</> : "GUARDAR API KEY"}
</button>
<p className="text-xs text-nf-grey-dim text-center">
La key se almacena solo en tu navegador (localStorage). Nunca se envía a terceros.
</p>
</div>
</div>
</div>
);
}
```

SECCIÓN 18

VERCEL (RECOMENDADO)

1. Push el repositorio a GitHub bajo tu cuenta YANKYFILMS.
2. Conecta el repo en vercel.com → New Project → Import Git Repository.
3. Framework Preset: Next.js (auto-detectado).
4. En Environment Variables añade: NEXT_PUBLIC_KLING_API_KEY y OPENAI_API_KEY.
5. Click Deploy. Listo. URL pública en 2 minutos.

DOCKER / GRANOSCAR SELF-HOSTED

```
# Dockerfile ya incluido en el repo base — solo ajustar:
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
ENV NEXT_PUBLIC_KLING_API_KEY=''
ENV OPENAI_API_KEY=''
COPY --from=builder /app/.next ./.next
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/public ./public
COPY --from=builder /app/package.json ./package.json
EXPOSE 3000
CMD ["npm", "start"]

# docker-compose.yml para GRANOSCAR
version: "3.8"
services:
nexframe:
build: .
ports:
- "3000:3000"
environment:
```

```
- NEXT_PUBLIC_KLING_API_KEY=${KLING_KEY}
- OPENAI_API_KEY=${OPENAI_KEY}
restart: unless-stopped

# Arrancar:
docker-compose up -d

# Coolify (recomendado para GRANOSCAR):
# Añade el repo en Coolify → Build Pack: Dockerfile → Deploy
```

SECCIÓN 19

Antes de entregar el proyecto como completado, verifica cada punto de esta lista. Si algún ítem no está al 100%, NO marques el proyecto como terminado.

REPOSITORIO

- [ ] Fork de Open Generative AI clonado con --recurse-submodules
- [ ] npm run setup ejecutado sin errores
- [ ] .env.local configurado con KLING_API_KEY y OPENAI_API_KEY
- [ ] npm run dev arranca sin errores en localhost:3000

API KLING

- [ ] src/lib/kling-client.js creado con todos los métodos
- [ ] Patrón submit → poll funcionando en generateTextToVideo
- [ ] Patrón submit → poll funcionando en generateImage
- [ ] Patrón submit → poll funcionando en generateLipSync
- [ ] generateSound funcionando con Kling Sound API
- [ ] uploadFile funcionando para subir imágenes/audio

DISEÑO

- [ ] tailwind.config.js actualizado con tokens nf-* completos
- [ ] Bebas Neue y Space Grotesk cargadas via next/font
- [ ] app/layout.js actualizado con fuentes y background nf-black
- [ ] Paleta dark red/black/gold aplicada consistentemente

STUDIOS

- [ ] Video Studio: T2V funcional con Kling (todos los modelos)
- [ ] Video Studio: I2V funcional cuando hay imagen subida
- [ ] Image Studio: T2I funcional con kolors-v1
- [ ] Sound Studio: generación de audio funcional
- [ ] Lip Sync Studio: imagen+audio → video funcional
- [ ] Cinema Studio: controles de cámara generan prompt correcto

- [ ] Documentary Studio: flujo 4 pasos completo funcionando
- [ ] Music Video Studio: flujo 4 pasos completo funcionando

DASHBOARD

- [ ] Sidebar con navegación completa funcional
- [ ] TopBar con indicador de API key y toggle sidebar
- [ ] ApiKeyModal guarda key en localStorage via Zustand
- [ ] Historial muestra generaciones con preview hover-play
- [ ] Store Zustand persistiendo entre recargas de página

API ROUTES

- [ ] app/api/script/generate/route.js generando guiones
- [ ] app/api/script/scenes/route.js extrayendo escenas
- [ ] app/api/script/musicvideo/route.js generando storyboards

DEPLOYMENT

- [ ] npm run build sin errores TypeScript/ESLint
- [ ] Variables de entorno configuradas en Vercel/Coolify
- [ ] App accesible en URL pública o localhost:3000
- [ ] Sin botones rotos, sin textos de placeholder

NEXFRAME FILMS — The Future of Filmmaking

Powered by Kling AI API | Built for YANKYFILMS